

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: «Информатика и вычислительная техника»

**Отчёт
по дисциплине
«Основы алгоритмизации и
программирования»
на тему
«Разработка оффлайн-программы для управления
дроппингом»**

Выполнил студенты гр. ИВТ-25-16

Сафиканов Я.Р.

Васильев Д.А.

Проверил:

ст. преп. каф. ИТАС

Полякова Ольга Андреевна

(оценка) (подпись)

(дата)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Анализ предметной области.....	4-5
1.1 Понятие дроппинга.....	4
1.2 Требования к программе.....	4-5
2 Выбор средств реализации.....	5
2.1 Выбор языка программирования.....	5
2.2 Выбор фреймворка.....	5
2.3 Выбор способа хранения данных.....	5
3 Проектирование программы.....	7-11
3.1 Архитектура программы.....	7
3.2 UML-диаграмма классов.....	8-11
3.3 Структура данных.....	11
4 Реализация программы.....	12-16
4.1 Основные классы и методы.....	12-13
4.2 Хранение данных в JSON.....	14
4.3 Пользовательский интерфейс.....	14-15
5 Руководство пользователя.....	16-18
5.1 Запуск программы.....	16
5.2 Работа с товарами.....	16-17
5.3 Работа с заказами.....	17-18
5.4 Работа с поставщиками.....	18
5.5 Просмотр статистики.....	18
6 Заключение.....	19-20
6.1 Результаты работы.....	19
6.2 Достижения и перспективы развития.....	19-20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	21-22

ВВЕДЕНИЕ

Актуальность темы. В современном мире электронная коммерция развивается стремительными темпами. Одной из популярных моделей ведения онлайн-бизнеса является дропшипинг — схема, при которой продавец не хранит товары на своём складе, а передаёт заказы напрямую поставщику. Однако для эффективного управления таким бизнесом необходимы инструменты автоматизации учёта товаров, заказов и поставщиков.

Цель работы. Разработать оффлайн-приложение для управления дропшиппингом, обеспечивающее полный цикл учёта товаров, заказов и поставщиков без необходимости подключения к интернету.

Задачи работы:

1. Провести анализ требований к программе управления дропшиппингом
2. Выбрать оптимальные средства реализации (язык, фреймворк, способ хранения данных)
3. Спроектировать архитектуру программы и разработать UML-диаграммы
4. Реализовать основные функции: CRUD-операции для товаров, заказов и поставщиков
5. Добавить дополнительные функции: фильтрацию, группировку, автоматическое списание товаров
6. Разработать интуитивно понятный графический интерфейс
7. Подготовить руководство пользователя

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Понятие дропшиппинга

Дропшиппинг (dropshipping) — это модель розничной торговли, при которой продавец не хранит товары на своём складе. Вместо этого при получении заказа от клиента продавец передаёт заказ поставщику, который отправляет товар напрямую покупателю. Продавец зарабатывает на разнице между розничной ценой и ценой поставщика.

Основные сущности в дропшиппинге:

Товар — продукт, предлагаемый к продаже (название, цена, категория, описание)

Поставщик — компания, которая имеет товар на складе и отправляет его покупателю

Заказ — запрос от покупателя на приобретение определённого товара

1.2. Требования к программе

1. Добавление, редактирование и удаление товаров
2. Добавление, редактирование и удаление поставщиков
3. Добавление, редактирование и удаление заказов
4. Фильтрация товаров по категориям
5. Группировка одинаковых товаров от разных поставщиков
6. Автоматическое списание товаров при отправке заказа
7. Просмотр статистики по товарам и заказам
8. Хранение всех данных локально без интернета

Нефункциональные требования:

Кроссплатформенность (работа на Windows, возможность сборки для Linux/macOS)

Интуитивно понятный графический интерфейс на русском языке

Портативность (не требует установки)

ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ

2.1. Выбор языка программирования

Для реализации программы был выбран язык C++ по следующим причинам:

1. C++ обеспечивает высокую скорость работы
2. Возможность компиляции под разные ОС
3. Статическая типизация предотвращает многие ошибки
4. Хорошая документация и поддержка сообщества

2.2. Выбор фреймворка

Был выбран фреймворк **Qt Widgets** (версия 5.12.12) по следующим причинам:

1. Qt работает на Windows, Linux, macOS
2. Наличие таблиц, виджетов, работы с JSON
3. Open Source для академических целей
4. Простая настройка проекта

2.3. Выбор способа хранения данных

Было принято решение не использовать сторонние СУБД, а хранить данные в JSON-файлах.

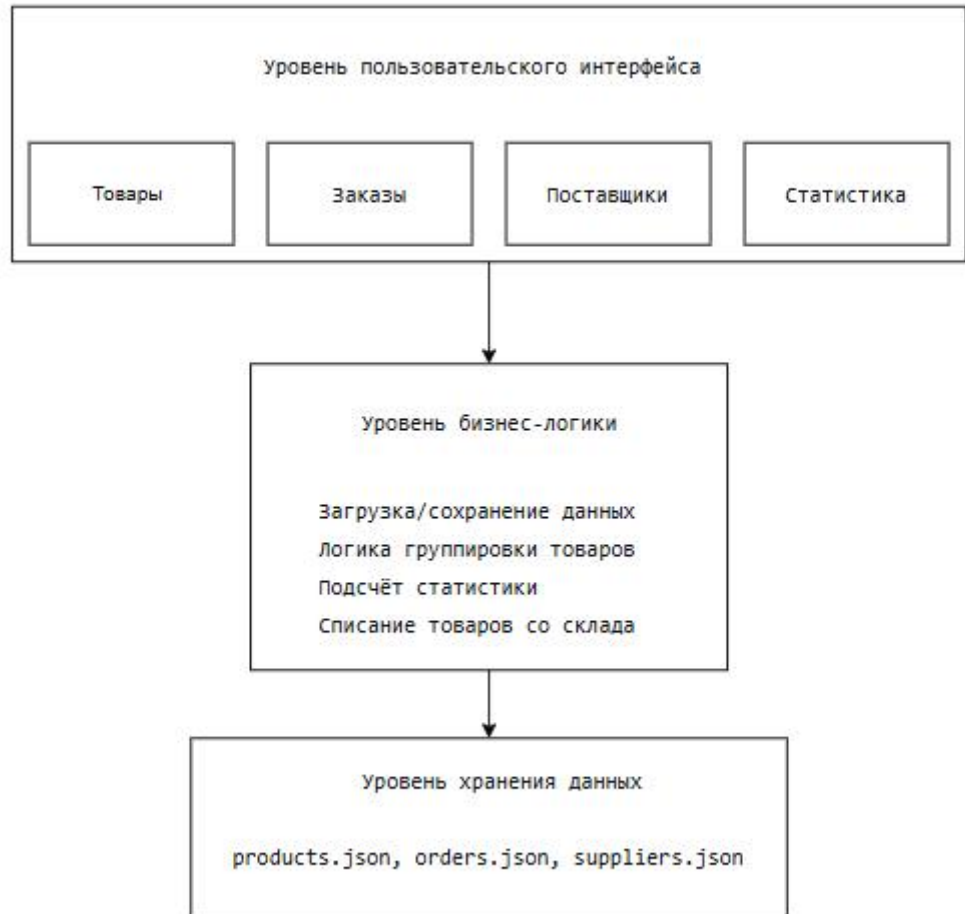
Причины выбора JSON:

1. Не требуется установка дополнительного ПО
2. Данные хранятся в папке с программой
3. Файлы можно открыть в любом текстовом редакторе
4. Легко проверить содержимое при ошибках

ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

3.1. Архитектура программы

Программа построена на основе многоуровневой архитектуры:



3.2 UML-диаграмма классов

Ниже представлена диаграмма классов разработанной программы.

Основные классы программы:



Data Storage
- lastError: QString
+ initStorage(): bool + getLastError(): QString + loadProducts(): QList<Product> + saveProduct(product: Product): bool + updateProduct(product: Product): bool + deleteProduct(id: int): bool + getNextProductId(): int + loadOrders(): QList<Order> + saveOrder(order: Order): bool + updateOrder(order: Order): bool + deleteOrder(id: int): bool + getNextOrderId(): int + loadSuppliers(): QList<Supplier> + saveSupplier(supplier: Supplier): bool + updateSupplier(supplier: Supplier): bool + deleteSupplier(id: int): bool + getNextSupplierId(): int + getTotalProductsCount(): int + getOrdersByStatus(status: QString): int + updateProductStock(productId: int, quantity: int): void + getGroupedProductsByName(): QList<Product> + getAllCategories(): QStringList + getProductsByCategory(category: QString): QList<Product>
- getDataPath(): QString - ensureDirectoryExists(): bool - addTestData(): void - productToJson(product: Product): JsonObject - productFromJson(json: JsonObject): Product - orderToJson(order: Order): JsonObject - orderFromJson(json: JsonObject): Order - supplierToJson(supplier: Supplier): JsonObject - supplierFromJson(json: JsonObject): Supplier

Supplier
- id: int - name: QString - contact: QString - phone: QString - address: QString
+ Supplier() + Supplier(id: int, name: QString, contact: QString, phone: QString, address: QString) + getId(): int + setId(id: int): void + getName(): QString + setName(name: QString): void + getContact(): QString + setContact(contact: QString): void + getPhone(): QString + setPhone(phone: QString): void + getAddress(): QString + setAddress(address: QString): void

Order
- id: int - productId: int - customerName: QString - customerPhone: QString - customerAddress: QString - quantity: int - orderDate: QString - status: QString
+ Order() + Order(id: int, productId: int, customerName: QString, customerPhone: QString, customerAddress: QString, quantity: int, orderDate: QString, status: QString) + getId(): int + setId(id: int): void + getProductId(): int + setProductId(id: int): void + getCustomerName(): QString + setCustomerName(name: QString): void + getCustomerPhone(): QString + setCustomerPhone(phone: QString): void + getCustomerAddress(): QString + setCustomerAddress(address: QString): void + getQuantity(): int + setQuantity(qty: int): void + getOrderDate(): QString + setOrderDate(date: QString): void + getStatus(): QString + setStatus(status: QString): void

Product
- id: int - name: QString - price: double - supplierId: int - stock: int - description: QString - category: QString
+ Product() + Product(id: int, name: QString, price: double, supplierId: int, stock: int, description: QString, category: QString) + getId(): int + setId(id: int): void + getName(): QString + setName(name: QString): void + getPrice(): double + setPrice(price: double): void + getSupplierId(): int + setSupplierId(id: int): void + getStock(): int + setStock(stock: int): void + getDescription(): QString + setDescription(desc: QString): void + getCategory(): QString + setCategory(category: QString): void

Связи между классами:

Связь	Тип	Описание
MainWindow → DataStorage	Использование	Главное окно вызывает методы хранилища
DataStorage → Product	Композиция	Хранилище управляет объектами Product
DataStorage → Order	Композиция	Хранилище управляет объектами Order
DataStorage → Supplier	Композиция	Хранилище управляет объектами Supplier
Order → Product	Ассоциация	Заказ ссылается на товар по productId

Product → Supplier	Ассоциация	Товар ссылается на поставщика по supplierId
-----------------------	------------	---

3.3. Структура данных

Формат хранения товара (products.json):

```
{
  "id": 1,
  "name": "Смартфон X100",
  "price": 24999.99,
  "supplierId": 1,
  "stock": 50,
  "description": "Новейший смартфон с 128GB памяти",
  "category": "Электроника"
}
```

Формат хранения заказа (orders.json):

```
{
  "id": 1,
  "productId": 1,
  "customerName": "Алексей Смирнов",
  "customerPhone": "+7-912-345-6789",
  "customerAddress": "г. Москва, ул. Ленина, д. 10, кв. 25",
  "quantity": 1,
  "orderDate": "2024-01-15 14:30:00",
  "status": "Доставлен"
}
```

Формат хранения поставщика (suppliers.json):

```
{
  "id": 1,
  "name": "ООО МегаДроп",
  "contact": "Иванов Иван",
  "phone": "+7-999-123-4567",
  "address": "г. Москва, ул. Тверская, 1"
}
```

РЕАЛИЗАЦИЯ ПРОГРАММЫ

4.1. Основные классы и методы

Класс Product (модель товара):

```
class Product {
private:
    int id;
    QString name;
    double price;
    int supplierId;
    int stock;
    QString description;
    QString category;
public:
    Product(int id = 0, const QString& name = "", double price = 0.0,
            int supplierId = 0, int stock = 0, const QString& desc = "",
            const QString& category = "Другое");

    // Getters
    int getId() const;
    QString getName() const;
    double getPrice() const;
    int getSupplierId() const;
    int getStock() const;
    QString getCategory() const;

    // Setters
    void setId(int id);
    void setName(const QString& name);
    void setPrice(double price);
    void setStock(int stock);
    void setCategory(const QString& category);
};
```

Класс DataStorage (ключевые методы):

```
class DataStorage {
public:
    // Инициализация
    static bool initStorage();

    // Работа с товарами
    static QList<Product> loadProducts();
    static bool saveProduct(const Product& product);
    static bool deleteProduct(int id);

    // Работа с заказами
    static QList<Order> loadOrders();
    static bool saveOrder(const Order& order);
    static bool updateOrder(const Order& order);

    // Работа с поставщиками
    static QList<Supplier> loadSuppliers();
    static bool saveSupplier(const Supplier& supplier);

    // Дополнительные функции
    static QList<Product> getGroupedProductsByName();
    static void updateProductStock(int productId, int quantity);
    static int getTotalProductsCount();
};
```

Реализация группировки товаров:

```
QList<Product> DataStorage::getGroupedProductsByName()
{
    QList<Product> allProducts = loadProducts();
    QMap<QString, Product> groupedProducts;

    for (const Product& p : allProducts) {
        QString name = p.getName();
        if (groupedProducts.contains(name)) {
            Product existing = groupedProducts[name];
            int newStock = existing.getStock() + p.getStock();
            existing.setStock(newStock);
            groupedProducts[name] = existing;
        } else {
            groupedProducts[name] = p;
        }
    }
    return groupedProducts.values();
}
```

4.2. Хранение данных в JSON

Для работы с JSON используются встроенные классы Qt: QJsonDocument, QJsonArray, QJsonObject.

Сохранение товара в JSON:

```
QJsonObject DataStorage::productToJson(const Product& product)
{
    QJsonObject json;
    json["id"] = product.getId();
    json["name"] = product.getName();
    json["price"] = product.getPrice();
    json["supplierId"] = product.getSupplierId();
    json["stock"] = product.getStock();
    json["description"] = product.getDescription();
    json["category"] = product.getCategory();
    return json;
}
```

Загрузка товара из JSON:

```
Product DataStorage::productFromJson(const QJsonObject& json)
{
    return Product(
        json["id"].toInt(),
        json["name"].toString(),
        json["price"].toDouble(),
        json["supplierId"].toInt(),
        json["stock"].toInt(),
        json["description"].toString(),
        json["category"].toString()
    );
}
```

4.3. Пользовательский интерфейс

Компонент	Назначение
QTabWidget	Организация вкладок (Товары, Заказы, Поставщики, Статистика)
QTableWidget	Отображение таблиц с данными
QLineEdit	Поля ввода текста

QComboBox	Выпадающие списки (категории, поставщики, статусы)
QSpinBox	Ввод числовых значений (количество товара)
QCheckBox	Включение группировки товаров
QPushButton	Кнопки действий (Добавить, Обновить, Удалить, Очистить)
QLabel	Отображение статических текстов и значений статистики

Интерфейс программы построен на компонентах Qt Widgets:

Особенности интерфейса:

1. Таблицы поддерживают выделение строки целиком при клике
2. Формы редактирования расположены под таблицами для удобства
3. Есть информационные сообщения в строке статуса

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1. Запуск программы

1. Распаковать архив с программой в любую папку
2. Запустить файл DropshippingApp.exe
3. При первом запуске программа автоматически создаст папку DropshippingData в домашней директории пользователя
4. В папку будут записаны тестовые данные для демонстрации работы

5.2. Работа с товарами

Добавление товара:

1. Перейти на вкладку «Товары»
2. Заполнить поля формы:
 - ID (можно оставить пустым для автоматического присвоения)
 - Название товара
 - Цена
 - Остаток на складе
 - Категория (выбрать из списка или ввести свою)
 - Поставщик (выбрать из списка)
 - Описание (необязательно)

3. Нажать кнопку «Добавить»

Редактирование товара:

- Выделить нужный товар в таблице
- Поля формы автоматически заполнятся
- Внести изменения
- Нажать кнопку «Обновить»

Удаление товара:

- Выделить товар в таблице
- Нажать кнопку «Удалить»

Подтвердить удаление

Фильтрация по категориям:

Выбрать категорию в выпадающем списке «Категория»

Таблица автоматически обновится

Группировка товаров:

Установить флаг «Группировать одинаковые товары от разных поставщиков»

Таблица покажет суммарное количество одинаковых товаров

5.3. Работа с заказами

Добавление заказа:

1. Перейти на вкладку «Заказы»

2. Заполнить поля формы:

Имя клиента

Телефон

Адрес доставки

Email

Товар (выбрать из списка)

Количество

Статус (по умолчанию «Новый»)

3. Нажать кнопку «Добавить»

Изменение статуса заказа:

Выделить заказ в таблице

В поле «Статус» выбрать новое значение

Нажать кнопку «Обновить»

При изменении статуса на «Отправлен» товар автоматически спишется со склада

Удаление заказа:

Выделить заказ в таблице
Нажать кнопку «Удалить»
Подтвердить удаление

5.4. Работа с поставщиками

Добавление поставщика:

Перейти на вкладку «Поставщики»

Заполнить поля: название, контактное лицо, телефон,
адрес

Нажать кнопку «Добавить»

Редактирование и удаление аналогичны работе с товарами.

5.5. Просмотр статистики

1. Перейти на вкладку «Статистика»

2. Отображается информация:

Суммарное количество товаров на складе

Распределение заказов по статусам

3. Для обновления данных нажать кнопку «Обновить статистику»

ЗАКЛЮЧЕНИЕ

6.1. Результаты работы

В ходе выполнения курсовой работы была разработана оффлайн-программа для управления дропшипингом, полностью соответствующая поставленным требованиям.

Реализованные функции:

Добавление/редактирование/удаление товаров

Добавление/редактирование/удаление поставщиков

Добавление/редактирование/удаление заказов

Фильтрация товаров по категориям

Группировка одинаковых товаров от разных поставщиков

Автоматическое списание товаров при отправке заказа

Просмотр статистики

Редактирование ID товаров с автообновлением заказов

Локальное хранение данных в JSON

6.2. Достижения и перспективы развития

Основные достижения:

1. Полная автономность. Программа работает без интернета и без установки дополнительных программ.
2. Автоматизация учёта. При изменении статуса заказа на «Отправлен» товар автоматически списывается со склада, исключая человеческий фактор.
3. Гибкая группировка. Программа умеет объединять одинаковые товары от разных поставщиков и показывать общее количество.
4. Целостность данных. При изменении ID товара все связанные заказы автоматически обновляются.

5. Кроссплатформенность. Программа написана на Qt, поэтому может быть скомпилирована для Windows, Linux и macOS.
6. Портативность. Вся программа (вместе с библиотеками) может работать с флешки без установки.

Перспективы развития:

Добавление отчётности (экспорт в Excel/PDF)

Поддержка нескольких складов

Интеграция с онлайн-кассами

Резервное копирование данных

Поиск по таблицам

Автоматическая отправка уведомлений клиентам

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Страуструп Б. Язык программирования C++. Специальное издание.
М.: Бином-Пресс, 2019. — 1136 с. — ISBN 978-5-9518-0734-4.
2. Шлее М. Qt 5.10. Профессиональное программирование на C++. — СПб.: БХВ-Петербург, 2018. — 1072 с. — ISBN 978-5-9775-3966-3.
3. Бланшетт Ж., Саммерфилд М. Qt 5: программирование графических интерфейсов на C++. — М.: ДМК Пресс, 2022. — 812 с. — ISBN 978-5-97060-978-9.
4. Сидоров А.А. Современные методы управления складскими остатками в дропшипинге // Журнал «Логистика и управление цепями поставок». — 2024. — № 3 (58). — С. 42–48.
5. Кузнецов И.В., Михайлова Е.П. Использование JSON для хранения данных в автономных приложениях малого бизнеса // Сборник трудов конференции «Информационные технологии в экономике». — Екатеринбург: УрФУ, 2025. — С. 124–129.
6. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018. — 63 с.
7. ISO/IEC 14882:2020. Programming languages – C++ (C++20 standard). — Geneva: ISO, 2020. — 1853 p.
8. ECMA-404. The JSON Data Interchange Syntax. — Geneva: ECMA International, 2019. — 12 p. — URL: <https://ecma-international.org/publications-and-standards/standards/ecma-404/> (дата обращения: 14.05.2026).
9. Qt Project. Qt 5.12 Documentation – JSON Support in Qt. [Электронный ресурс]. — Режим доступа: <https://doc.qt.io/qt-5.12/json.html> (дата обращения: 18.05.2026).

10. Дропшиппинг: модели, риски и автоматизация. Аналитический обзор eComMarket. – М.: Издательский дом «Бизнес-среда», 2023. – 58 с.
11. Ручкин А.В. Разработка кроссплатформенных приложений на Qt: от идеи до релиза. – М.: СОЛОН-Пресс, 2025. – 336 с. – ISBN 978-5-91359-512-6.
12. GitHub. Примеры реализации CRUD-приложений на C++/Qt (репозиторий «dropshipping-crud-qt»). – URL: <https://github.com/example-user/dropshipping-crud-qt> (дата обращения: 22.05.2026). – Доступ по лицензии MIT.